Asynchronous Data Gateway for LoRa-to-Mobile Integration in Disaster Mesh Networks (2026)

Supervisors: Dr Lilian Wang, Dr Wai Peng Wong
Ming Dong Teh and Jing Hong Hiew

Abstract: ***Natural disasters routinely disable cellular infrastructure at the precise moment reliable communication is most critical. In Malaysia and Southeast Asia, the AS923 LoRa spectrum offers a viable low-power fallback, yet no production-ready, open-source Android application exists to act as a true asynchronous middleware gateway between a commodity LoRa transceiver and a mobile device user interface over USB serial. This paper presents the DEC-AI LoRa Gateway: a custom Android middleware application that implements (1) a bespoke binary mesh protocol optimised for sub-50-byte maximum transmission units (MTUs); (2) asynchronous, state-machine-driven serial communication over USB OTG targeting the Heltec V3 ESP32 (SX1262 transceiver) via a Xiaomi 12T running Android 12; and (3) selective-repeat automatic repeat request (ARQ) fragmentation for reliable transmission of structured disaster data including GPS coordinates, status reports, and alert messages under AS923 duty-cycle constraints. The proposed architecture is benchmarked against Meshtastic [6], MeshCore , LoRaWAN, Reticulum, and the recently published Stratos system [8], demonstrating that building the disaster-specific protocol layer atop the stable Meshtastic Android transport stack offers a superior convergence of reliability, bandwidth efficiency, and research novelty. Empirical evaluation targets packet delivery ratio, throughput, latency, and power consumption across indoor and outdoor line-of-sight test scenarios at the Monash University campus in Malaysia.***

Index Terms: **LoRa, AS923, disaster communication, Android middleware, USB OTG, ARQ, binary protocol, Heltec V3, mesh network, Malaysia.**

I. Introduction and Literature Review

THE failure of telecommunications infrastructure during natural disasters is a well-documented and persistent problem. Floods in Malaysia, earthquakes across Southeast Asia, and typhoons throughout the Pacific annually demonstrate that cellular towers and internet backhaul are among the first systems to fail, yet they are precisely the systems upon which modern emergency coordination depends. This creates a communication void between affected civilians, first responders, and coordination centres at the most critical window of a disaster response.

A. Background and Motivation

Long Range (LoRa) radio has emerged as a technically compelling fallback medium for such scenarios. Operating in the unlicensed AS923 band designated for IoT use in Malaysia (923.2–924.6 MHz), LoRa provides kilometre-scale range with sub-milliamp receive currents and does not require any licensed spectrum or network infrastructure [1]. Foundational empirical work by Lee and Ke [5] validated LoRa mesh at campus scale: a 20-node deployment achieved a packet delivery ratio (PDR) of 99.99% for periodic traffic, with multi-hop latencies of 2.28 s, 4.60 s, and 6.75 s for one-, two-, and three-hop paths respectively, without any cellular backbone.

Despite this validated physical-layer capability, a critical software gap persists at the Android middleware layer. No production-ready, open-source Android application exists that functions as a fully asynchronous gateway between a commodity LoRa transceiver connected via USB serial OTG and a mobile device user interface, implementing a custom binary protocol with built-in ARQ and duty-cycle enforcement. This project directly addresses that gap.

B. Literature Review

1) LoRa Mesh for Disaster Communications

The suitability of LoRa mesh for post-disaster communication has been corroborated across multiple deployment studies and system architectures. Sciullo, Trotta, and Di Felice [2] introduced LOCATE, a wearable IoT system pairing a LoRa transceiver with an Android application via Bluetooth Low Energy (BLE) for opportunistic, multi-hop alert dissemination in post-disaster scenarios. While LOCATE confirmed that a mobile-phone-centric LoRa architecture is operationally feasible, BLE introduces additional pairing complexity and a wireless link that is vulnerable to interference in dense urban rubble environments; this project targets USB OTG serial as a more deterministic and robust alternative.

Trotta et al. [3] conducted a substantive analysis of LoRa-based peer-to-peer mesh architecture and identified the single point-of-failure inherent in LoRaWAN's star-of-stars topology. Their finding that all existing LoRa mesh overlays at the time still partially required gateways or internet backhaul which renders them unsuitable for fully offline disaster scenarios. This remains the primary architectural motivation for this project's gateway-free design goal.

Carrillo et al. [4] published what constitutes the most technically relevant 2025 contribution: a gateway-free LoRa mesh system on ESP32 implementing neighbour-based routing, hop-by-hop acknowledgements, Listen-Before-Talk (LBT), and duplicate suppression via Alternate Link Triggering (ALT). This work empirically confirms that lightweight self-healing mechanisms are achievable on commodity microcontrollers, directly informing the ARQ and routing design of this project's protocol specification.

2) Existing Mesh Protocol Ecosystems

Meshtastic (v2.7.8, April 2025) represents the most mature open-source LoRa mesh platform, supporting Android, iOS,

macOS, and Linux clients communicating with ESP32/nRF52-based nodes via Bluetooth, USB serial, or TCP [6]. It encodes all on-air packets using Protocol Buffers and implements a flooding-based mesh with configurable hop limits. Its Android application uses the usb-serial-for-android library with a ProbeTableProvider for device-specific driver mappings, an architectural pattern this project directly reuses [10]. The principal limitation of Meshtastic from the perspective of this research is that it requires proprietary Meshtastic firmware on all nodes and provides no mechanism for custom binary protocol overlays or application-layer duty-cycle enforcement. These limitations define the research gap this project occupies.

MeshCore is a lightweight C++ library targeting disaster recovery that emphasises minimal overhead and an open firmware architecture [6]. It provides a useful comparative benchmark for protocol overhead analysis, but its absence of a mature Android middleware layer and the lack of a documented USB OTG integration path means it does not address the Android-gateway dimension of this research.

The recently published Stratos system by Sai and Sujini [8] introduces a decentralized LoRa mesh architecture combining a Raspberry Pi gateway, Wi-Fi captive portal, BLE crowd estimation, and TinyML-based message prioritisation for civilian SOS messaging. While Stratos demonstrates sophisticated edge-intelligence integration, its architectural dependency on a Raspberry Pi gateway and Python-based backend means it is not a native Android solution. Civilians must interact through a web captive portal rather than a direct Android application, and the system cannot function as a device-carried, hand-held gateway without the Raspberry Pi hardware. This represents a significant operational constraint in rapid-onset, highly mobile disaster response scenarios where a responder carrying only a smartphone and a USB-connected LoRa node needs autonomous communication capability.

LoRaWAN's star-only topology, mandatory join procedure, and 14-byte minimum MAC overhead make it fundamentally unsuitable for direct Android-to-gateway peer-to-peer messaging [1]. Reticulum (RNS) offers mesh routing with end-to-end encryption but imposes cryptographic metadata overhead per packet that is incompatible with 50-byte MTUs at SF12 (0.3 kbps) under AS923 duty-cycle constraints.

3) Binary Serialisation for Constrained IoT

Shvaika et al. [9] conducted a comparative benchmarking study of serialisation formats across IoT ecosystems, confirming that binary formats including Protocol Buffers and Avro outperform JSON by 60–80% in sensor telemetry workloads. For GPS latitude/longitude, timestamp, or status reports, a JSON encoding yields 78–90 bytes versus 13 bytes for a compact binary struct. At SF12 (0.3 kbps) under AS923's 1% duty-cycle constraint, this yields approximately 102 binary messages per hour versus 15 JSON messages, a difference that may be operationally decisive in a disaster scenario.

Structured packet scheduling for ground-based LoRa IoT networks, including TDMA solutions addressing the contention and duty-cycle challenges that directly constrain this project's transmission scheduler, has been surveyed by Faiza et al. for the Malaysian IEEE MICC 2025 context [10].

4) Android Asynchronous Serial Communication

The usb-serial-for-android library (v3.8.x, 2024 - 2025) is the most actively maintained Android USB serial solution, supporting FTDI, CDC-ACM, CP210x, PL2303, and CH340/CH341 chipsets; the latter being the chipset used on the Heltec V3 [13]. The Xiaomi 12T running Android 12 (MIUI/HyperOS) supports USB OTG via its USB 2.0 Type-C port, confirmed by hardware specification [14]. Google's official Android documentation designates Kotlin Coroutines as the recommended asynchronous programming model for Android, with Kotlin Flows providing reactive streaming from background serial I/O coroutines to the UI layer [7], precisely the architectural model adopted by this project's gateway state machine.

5) Gap Analysis

Table I synthesises the comparative analysis of related protocols and systems. The critical gaps are: (1) no existing system combines a native Android application, USB OTG serial as the primary transport, a custom binary protocol with built-in ARQ and fragmentation, and AS923 duty-cycle enforcement in a single open-source middleware architecture; (2) Meshtastic is the closest technically, but requires proprietary firmware on all nodes and lacks duty-cycle enforcement in the Android client are the specific gaps this project addresses; (3) Stratos demonstrates the value of edge intelligence in disaster communication but requires a Raspberry Pi gateway, precluding use as a hand-carried smartphone-native solution.

TABLE I
COMPARATIVE ANALYSIS OF RELATED PROTOCOLS AND SYSTEMS

System / Protocol	Topology	Android	USB Serial	Binary Encoding	Offline Mesh	Limitation
LoRaWAN	Star-only	Via gateway	No	Yes (MAC)	No	Needs cloud; no multi-hop
MQTT-SN	Broker-based	Via IP	No	Partial	No	Requires IP/UDP substrate
Reticulum (RNS)	Mesh	No native	Limited	Yes	Yes	Heavy crypto; not mobile-native
Meshtastic	Flood mesh	Yes (BLE/USB)	Yes	Protobuf	Yes	Proprietary firmware on all nodes
MeshCore	Multi-hop	Yes	Partial	Yes	Yes	C++ only; no Android middleware
LOCATE (Sciullo 2021)	Multi-hop	Yes (BLE)	No	Partial	Yes	Research prototype; BLE bridge only
Stratos (Sai 2026)	LoRa mesh	No (RPi)	No	Partial	Yes	RPi gateway; no Android native
This Project (DEC-AI)	Flood mesh	Yes (USB OTG)	Yes (primary)	Binary/Protobuf	Yes	Target — to be validated

Table I: Comparative analysis of related protocols and systems against the proposed DEC-AI LoRa Gateway.

C. Research Question

The gap identified by the literature review leads to the following research question:

> *Can a custom asynchronous Android middleware application, communicating with a Heltec V3 ESP32 (SX1262) via USB OTG serial and implementing a bespoke binary mesh protocol with selective-repeat ARQ, fragmentation, and AS923 duty-cycle enforcement, achieve a packet delivery ratio of ≥95% and a throughput of ≥50 binary messages per hour under realistic disaster channel conditions in Malaysia?*

This question defines the target platform, hardware, region, reliability, and throughput metrics, all of which are measurable through the evaluation plan described in Section 2G. Its significance lies in determining whether the DEC-AI protocol design is sufficiently reliable and practical for disaster-response deployment in Malaysia.

D. Problem Statement

When natural disasters disable cellular infrastructure in Malaysia and Southeast Asia, affected communities lose access to the communication systems they depend on for requesting rescue, reporting locations, and coordinating emergency response. While LoRa radio technology operates independently of this infrastructure and has been empirically validated for multi-kilometre, battery-powered mesh communication, the software gap at the Android middleware layer means that commodity smartphones cannot be used as autonomous LoRa gateways without either adopting Meshtastic's proprietary firmware ecosystem, deploying a fixed Raspberry Pi gateway as in Stratos [8], or accepting the heavyweight cryptographic overhead of Reticulum.

Specifically, the problem is the absence of an open-source Android application that addresses the gaps identified in the Gap analysis and enforces AS923's 1% duty-cycle constraint in software on the Android device.

The impact of this gap is concrete: without such a system, a first responder carrying only a Xiaomi 12T and a Heltec V3 LoRa node has no way to send structured GPS-tagged alerts across a multi-hop LoRa mesh to other responders outside cellular range. This research directly addresses that gap.

E. Aim and Objectives

The aim of this project is to design, implement, and empirically evaluate the DEC-AI LoRa Gateway: a custom Android middleware application enabling reliable, duty-cycle-compliant, structured disaster data communication over a LoRa mesh network via USB OTG serial, without dependence on proprietary node firmware or fixed network infrastructure.

The following objectives operationalise this aim:

I. Design and specify a custom binary mesh protocol for sub-50-byte MTUs, including a 13-byte fixed header, selective-repeat ARQ with NACK bitmap, and HELLO/ROUTE discovery messages.

II. Develop an Android foreground service implementing USB OTG serial communication with the Heltec V3 CH340/CH341 chipset using usb-serial-for-android v3.8.x [15], with SLIP framing and CRC-16 validation on all frames.

III. Implement a Kotlin Coroutines and Flows-based asynchronous state machine integrating the ARQ engine, fragment assembler, and AS923 duty-cycle scheduler, meeting the <1% duty-cycle and 400 ms dwell-time regulatory constraints.

IV. Empirically evaluate the system against three performance metrics: packet delivery ratio (target: ≥95%), message throughput (target: ≥50 binary messages/hour at SF12), and Android battery drain (mAh/hour) across different scenarios.

II. Methodology

A. Research and Design

This project adopts a design-science research methodology, proceeding through four sequential phases: literature review and hardware validation (Phase 1), protocol specification and architecture design (Phase 2), Android application development and unit testing (Phase 3), and empirical RF performance evaluation (Phase 4). The methodology is grounded in empirical validation, in which protocol design decisions are directly informed by measurable airtime and PDR metrics under realistic channel conditions.

The decision to build the DEC-AI disaster layer atop the Meshtastic Android transport stack is a deliberate architectural choice due to project time constraints. Meshtastic provides a battle-tested BLE/USB transport, RadioTransport interface, and MeshService foreground service lifecycle that would save several months of development.

B. System Architecture

TABLE II
OSI Layer Mapping for the DEC-AI LoRa Gateway Stack

OSI	OSI Layer	DEC-AI Component	Protocol / Technology	Hardware / Library
7	Application	DEC-AI Gateway App UI (Compose)	GPS, Alert, Status Message Types	Xiaomi 12T, Jetpack Compose
6	Presentation	Protobuf / Binary Serialiser	Protocol Buffers, MessagePack	protobuf-kotlin, msgpack-java
5	Session	ARQ Session Manager (Coroutines)	Selective-Repeat, ARQ Seq numbers	Kotlin Coroutines StateFlow
4	Transport	Fragment Engine (Kotlin Flows)	Custom fragmentation CRC-16	Kotlin Flows, usb-serial-for-android
3	Network	Mesh Routing (HELLO / ROUTE)	Custom binary mesh, protocol Flooding	Android Foreground Service, Kotlin
2	Data Link	Serial Framing SLIP encoding	SLIP, CRC-16 per frame	usb-serial-for-android v3.8.x, CH34x
1	Physical	USB OTG → Heltec V3 → SX1262	LoRa AS923 USB, CDC-ACM / CH34x	Heltec V3 (SX1262), U.FL-SMA Antenna

Table II: OSI layer mapping for the DEC-AI LoRa Gateway stack. Layers 1–2 are implemented in SX1262 hardware and Heltec V3 firmware; Layers 3–7 are implemented in the Android application.

C. Custom Binary Mesh Protocol Design

The DEC-AI binary mesh protocol is designed for operation within AS923 airtime budgets. At SF12 (125 kHz), the maximum LoRa payload is 255 bytes but duty-cycle-compliant airtime constrains the effective MTU to 50 bytes. The protocol adopts a 13-byte fixed header, leaving 40 bytes for application payload like GPS(4B lat + 4B lon + 4B timestamp + 1B status = 13B), an alert message (type byte + text up to 39B), or a fragmented segment of a longer message.

SLIP (Serial Line Internet Protocol) encoding is applied at the data-link layer to delimit packet boundaries on the USB serial stream. The SYNC field (0xAA55) provides an additional, protocol-level synchronisation marker that enables the SLIP decoder to resynchronise after partial frame corruption.

D. Selective-Repeat ARQ Engine

Reliable delivery of ALERT and GPS messages is provided by a selective-repeat ARQ engine. The transmitter maintains a window of unacknowledged packets identified by SEQ_NUM. On transmission, a 2000 ms acknowledgement timer is started

per packet. If an ACK (Acknowledgement from receiver) is received before timeout, the sequence number is retired from the window. If a NACK (Not Acknowledgement) is received, or the timeout expires, the specific packet is retransmitted. A maximum retransmission count of three is enforced before the packet is declared failed and a TxFailed event is emitted. The ARQ engine transitions through the IDLE → CONNECTING → HELLO_SENT → ACTIVE → RETRANSMIT / ERROR state machine shown in the project's FSM specification.

The NACK bitmap mechanism allows the receiver to report multiple missing fragments in a single ACK/NACK packet, reducing the overhead of individual per-fragment acknowledgements in multi-fragment message scenarios; an efficiency borrowed from the design pattern identified in Carrillo et al. [4].

E. AS923 Duty-Cycle Scheduler

The AS923 band plan for Malaysia imposes a 1% duty-cycle limit and a 400 ms maximum dwell time per sub-channel [13]. To enforce this, the DEC-AI DutyCycleScheduler implements these constraints in Android software. The scheduler maintains a per-channel airtime ledger: each transmitted packet's airtime is computed from its byte length and spreading factor, deducted from the available budget, and the channel is marked blocked until the budget recovers to the 1% threshold.

F. Android Application Architecture

The Android application is structured into four subsystems following the UML component model. The USB Serial Service is an Android Foreground Service that owns the USB OTG lifecycle, instantiates the appropriate usb-serial-for-android driver, applies SLIP framing, and emits validated ByteArray frames as a Kotlin Flow. The Protocol Engine subscribes to this flow, runs the ARQ state machine, manages the DutyCycleScheduler, and exposes a StateFlow of GatewayUiState to the presentation layer. The UI Layer, implemented in Jetpack Compose, observes GatewayUiState and renders the alert feed, GPS map (using offline OSM tiles), node status board, and duty-cycle meter. The architecture follows unidirectional data flow (UDF): all user actions flow downward through GatewayViewModel to the Protocol Engine, and all state changes flow upward via StateFlow.

The decision to architect the duty-cycle scheduler and ARQ engine in the Android domain layer rather than delegating to Heltec V3 firmware is a deliberate design choice that keeps the research contribution in Android software, makes the scheduler independently testable with Android unit tests, and removes the dependency on any particular firmware version on the ESP32.

G. Experimental Design

Performance evaluation proceeds across three scenarios emulating realistic operating conditions:

I. **Scenario A: Baseline (line-of-sight, 10–200 m):** PDR, throughput, and end-to-end latency measured at SF7 and SF9. A minimum of 100 message transmissions per configuration is required for statistical significance. Different message types are evaluated.

II. **Scenario B: Interference simulation:** A second Heltec V3 transmitting at 50% channel duty simulates collision conditions representative of a dense post-disaster RF environment. ACK/NACK bitmap efficiency (ratio of NACKs resolved without full retransmission) and retransmission rate are the primary metrics.

III. **Scenario C: Power consumption profiling:** Xiaomi 12T battery drain (mAh/hour) during continuous gateway operation is measured using Android Battery Historian and compared against the device idle baseline to quantify the energy cost of the foreground USB serial service.

All experiments use the Heltec V3 (SX1262, AS923 band) connected to the Xiaomi 12T (Android 12) via USB OTG Type-C.

H. Initial Results and Observations

Testing was limited as the hardware components were only received in week 11. Initial hardware testing confirmed the feasibility of using the Heltec V3 as a low-cost LoRa communication device.

REFERENCES

[1] O. Alsayyed, A. Al-Fuqaha, and Q. Abu Al-Haija, "Review of Technologies, Routing Protocols, and Architecture for Disaster Area Networks," IEEE Access, vol. 13, pp. 91129–91160, 2025. DOI: 10.1109/ACCESS.2025.3570733

[2] L. Sciullo, A. Trotta, and M. Di Felice, "LOCATE: A LoRa-based Mobile Emergency Management System for Infrastructure-Independent Search and Rescue Operations," Ad Hoc Networks, Elsevier, vol. 120, p. 102557, 2021.

[3] A. Trotta, M. Di Felice, F. Montori, K. R. Chowdhury, and L. Bononi, "LAURA: LoRa-Based UAV Assisted Robust Agriculture Architecture," MDPI Sensors, vol. 21, no. 13, p. 4314, 2021.

[4] J. Carrillo, R. Acosta-Coll, A. Perez-Diaz, and Y. Tomas-Grau, "Gateway-Free LoRa Mesh on ESP32: Neighbour-Based Routing with Hop-by-Hop ACKs and Listen-Before-Talk," MDPI Sensors, vol. 25, no. 19, p. 6036, 2025. DOI: 10.3390/s25196036

[5] W.-Y. Lee and Y.-Y. Ke, "A Monitoring System for Large-Area IoT Using Low-Power LoRa Mesh Networks," IEEE Transactions on Instrumentation and Measurement, vol. 67, no. 9, pp. 2177–2187, 2018.

[6] Meshtastic Project, "Meshtastic Official Documentation v2.7.8," 2025. [Online]. Available: https://meshtastic.org/docs

[7] Android Developers, "Kotlin Coroutines on Android," Google LLC, updated March 2026. [Online]. Available: https://developer.android.com/kotlin/coroutines

[8] K. T. Sai and G. N. Sujini, "Stratos: A Resilient LoRa Mesh Communication Network for Disaster Zones," International Journal on Science and Technology (IJSAT), vol. 17, no. 2, pp. 1–23, April–June 2026.

[9] A. Shvaika, O. Veres, and M. Beshley, "Dynamic Data Serialization: Performance of JSON vs. Binary Formats in IoT Ecosystems," Journal of Edge Computing, vol. 3, no. 2, 2024.

[10] Z. Faiza et al., "Structured Packet Scheduling for Ground-Based LoRa IoT Networks: Trends, Challenges and TDMA Solutions," in Proc. IEEE MICC, Malacca, Malaysia, 2025, pp. 1–6. DOI: 10.1109/MICC.2025.11210984

[11] Memfault, "Event Serialization for Constrained IoT: Protobuf and MessagePack for LoRa and BLE," 2024. [Online]. Available: https://docs.memfault.com

[12] DeepWiki, "USB Serial Communication: Meshtastic-Android Architecture," 2026. [Online]. Available: https://deepwiki.com

[13] LoRa Alliance, Inc, "RP002-1.0.2 LoRaWAN® Regional Parameters," 2020 https://lora-alliance.org/wp-content/uploads/2020/11/RP_2-1.0.2.pdf

[14] Semtech Corporation, "SX1262 Sub-GHz Radio Transceiver — Datasheet Rev 2.1," Semtech, Camarillo, CA, 2019.

[15] mik3y, "usb-serial-for-android v3.8.x," GitHub Repository, 2024. [Online]. Available: https://github.com/mik3y/usb-serial-for-android